```
Output:
calling containsLovely() ...
testing a Dom\Element ...
Found 1 matching node(s)
```

We can see from the output that `Dom\XPath::query()` called our anonymous function and passed an `array` containing a single `Dom\Element` instance (actually a sub-child `Dom\HTMLElement` instance) – a `Dom\NodeList` might have been more appropriate in my humble opinion, but you can't have everything. This element's text content contains the sub-string `"lovely"`, so we return true, which means the predicate in our XPath expression is true and we get our match on the `title` child of `head`. Please note the namespacing of the HTML elements as well as the PHP function string.

You might at this point be wondering why our anonymous function has a union data type (`array|string`) for its single parameter. I'm glad you asked, because it allows us to do this…

```
$result = $xpath->query(
    '//ns:head/ns:title[php:functionString("containsLovely", .)]'
);
print "Found {$result->length} matching node(s)" . PHP_EOL;
```

```
Output:
calling containsLovely() ...
testing a string ...
Found 1 matching node(s)
```

… which demonstrates a handy shortcut, `php:functionString()`, which will extract the text content of all the nodes in the current context, rather than passing the array of nodes and making you do it yourself. Notice also that I've tweaked the rest of the expression by moving the `ns:title` localization step out of the `php:functionString()` call and replacing it with an abbreviated `self::` step. It's functionally identical to the previous form and just a matter of preference (keeping predicate sub-expressions as clean as possible).

If, for some reason, you'd prefer your registered function to exist in a namespace other than `php`, then you can use `Dom\XPath::registerPhpFunctionNS()` to do something like this: